

Desarrollo de Aplicaciones: Clase 1

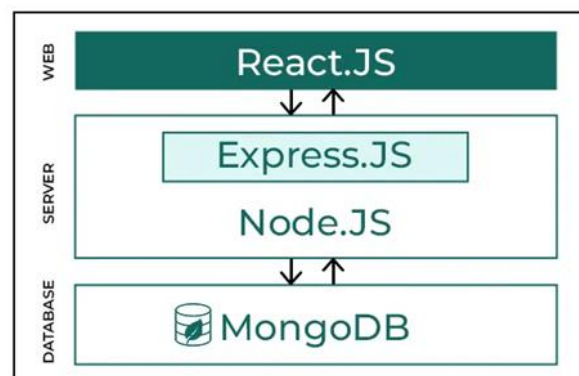
	BACK	FRONT
LENGUAJE	TypeScript/JavaScript	TypeScript/JavaScript
FRAMEWORK	Express	React
DataBase	Mongo DB (No Relacional - Documentos JSON)	

DB No Relacional:

- Tienen la capacidad de guardar datos de forma no estructurada a priori.
- Ejemplos: MogoDB, Cassandra y Redis, entre otras.
- Cada una esta orientada a resolver una problemática en particular.

Express ----> escrito en JavaScript ----> para ejecutarlo tenemos ----> NodeJs

How MERN Stack Works?



Cronograma de la materia:



NodeJS:

- Tiene que ver con JavaScript
- Ejecuta JS

Engine de ejecución de JS:

- NodeJS -
- Browser - DOM

Para ejecutar Node en la terminal usamos el comando: `node`

REPL ---> Read Eval Print Loop. Con Node podemos tener un REPL en nuestra maquina.

Repaso JS

Tipos de datos:

- **Number:** Para números de cualquier tipo, ya sean enteros o de punto flotante
- **BigInt:** Para números enteros de longitud arbitraria
- **String:** Para cadenas, que pueden tener cero o más caracteres
- **Boolean:** Para valores de verdadero o falso: `true` / `false`
- **Null:** Para valores desconocidos
- **Undefined:** Para valores no asignados

- **Symbol:** Para identificadores únicos
- **Object:** Para estructuras de datos complejas

Para declarar variables:

- **var:** no recomienda.
- **let:** ideal para declarar variables que luego van a cambiar.
- **const:** útil para ponerle nombre a tags que no van a cambiar. Si se intenta modificar se obtiene un error.

for in: El bucle *for in* itera sobre las propiedades de un objeto. Permite acceder a cada clave o al nombre de la propiedad de un objeto. También puede iterar sobre las propiedades de un array, pero no se recomienda. *for in* es principalmente adecuado para objetos.

(En REPL)

```
for (const e in {a: 10, b: 20, c: 30}) { console.log(e) }
a
b
c
```

for of: a diferencia del *for in*, este itera sobre los valores. Funciona para arrays, pero no para objetos.

(En REPL)

```
for (const e of {10, 20, 30}) { console.log(e) }
10
20
30
```

Hay dos maneras de importar en JS:

- CommonJS (Node)

Require/exports

```
JS pruebajs X
JS pruebajs > ...
1  const personas = require('./personas.js');
2
3  for (const element of personas) {
4    console.log('Hola ${element.nombre}, tu apellido, ${element.apellido} es muy conocido')
5  }
```

```
JS personas.js X
1  const personas = [
2    {
3      nombre: 'Pedro',
4      apellido: 'Pérez'
5    },
6    {
7      nombre: 'Juan',
8      apellido: 'González'
9    }
10 ]
11
12 const edadPromedio = 35;
13
14 module.exports = {
15   personas: personas,
16   edadPromedio: edadPromedio
17 }
18
```

(En REPL)

```
node prueba.js
Hola Pedro, tu apellido, Pérez es muy conocido
Hola Juan, tu apellido, Gonzáles es muy conocido
```

- ESM (estándar)

Import/export

IMPORTANTE: Cambiar la extensión de los archivos a .mjs

```
JS prueba.mjs X
JS prueba.mjs > ...
1 import { personas } from './personas';
2
3 for (const element of personas) {
4   console.log('Hola ${element.nombre}, tu apellido, ${element.apellido} es muy conocido')
5 }
```

```
JS personas.mjs X
JS personas.mjs > ...
1 export const personas = [
2   {
3     nombre: 'Pedro',
4     apellido: 'Pérez'
5   },
6   {
7     nombre: 'Juan',
8     apellido: 'Gonzáles'
9   }
10 ]
11
12 export const edadPromedio = 35;
```

¿Para qué se utiliza npm?

npm (Node Package Manager) es un gestor de paquetes para los proyectos de Node.js

Algunos comandos NPM son:

- **npm install:** Instala un paquete
- **npm init:** Construye el andamiaje de un proyecto

Al crear un nuevo proyecto con npm init, se lanzará un asistente que tras algunas preguntas, crea un archivo llamado package.json en la carpeta raíz del proyecto, donde coloca toda la información que se conoce sobre el mismo. Este archivo es un simple fichero de texto, en formato JSON que incorpora a través de varios campos información muy variada.

Este fichero tiene una estructura definida muy concreta, y a través de sus campos se puede guardar y recuperar información muy útil.

Tipo	Campo	Descripción
STRING	name	Nombre del proyecto, librería o paquete. Se recomienda que coincida con el repositorio.
STRING	version	Versión del paquete. Generalmente se utiliza semver
STRING	description	Descripción breve del paquete o proyecto.
STRING	type	Tipo de sistema de módulos a utilizar. Con <code>module</code> usa ESM, en caso contrario usa CommonJS.
STRING	main	Punto de entrada del proyecto. Suele ser <code>index.js</code> (node) o <code>index.html</code> (browser).
STRING	module	Idem al anterior, pero respecto a ES Modules en lugar de CommonJS.
OBJECT	scripts	Colección de scripts del proyecto
ARRAY	keywords	ARRAY de STRING con palabras clave relacionadas con el proyecto. Útil en búsquedas.
STRING	author	Nombre del autor del paquete o un ARRAY con <code>name</code> , <code>email</code> y/o <code>url</code> .
STRING	license	Tipo de licencia del paquete o proyecto. Por defecto, ISC .
OBJECT	dependencies	Colección de paquetes para producción y la versión instalada.
OBJECT	devDependencies	Colección de paquetes para desarrollo y la versión instalada.
STRING	homepage	URL de la página principal del paquete.
OBJECT	repository	URL del repositorio. Se debe indicar <code>type</code> (git, svn...) y <code>url</code> (ruta).
OBJECT	bugs	Objeto con campo <code>url</code> con la URL de la página de issues del proyecto.

Código asíncrono

Paradigma de programación que permite que varias tareas se ejecuten simultáneamente.

Características

- Permite que un programa trabaje en varias tareas al mismo tiempo
- Permite que un programa inicie una tarea de larga duración y siga respondiendo a otros eventos
- Permite dividir proyectos grandes y complejos en tareas más pequeñas

Técnicas para programar asíncronamente

- **Callbacks:** Son funciones que se pasan como argumento a otra función y se ejecutan después de que se haya completado la ejecución de la primera función.
- **Promises:** Son marcadores de posición para un valor o acción futura.
- **Async/await:** Es otra técnica para programar asíncronamente en JavaScript.

Callback hell es un problema que ocurre cuando se anidan funciones de devolución de llamada, dificultando la lectura y mantenimiento del código.

Como evitar callback hell:

- **Promises:** estandariza, pero sigue habiendo un lío de callbacks.
- **Promises con async await:** se parece mucho más al código sincrónico.

TypeScript

Para instalar --->

```
npm install typescript --save-dev
npm install @types/node --save-dev
npx tsc --init --rootDir src --outDir dist
npm tsc
```